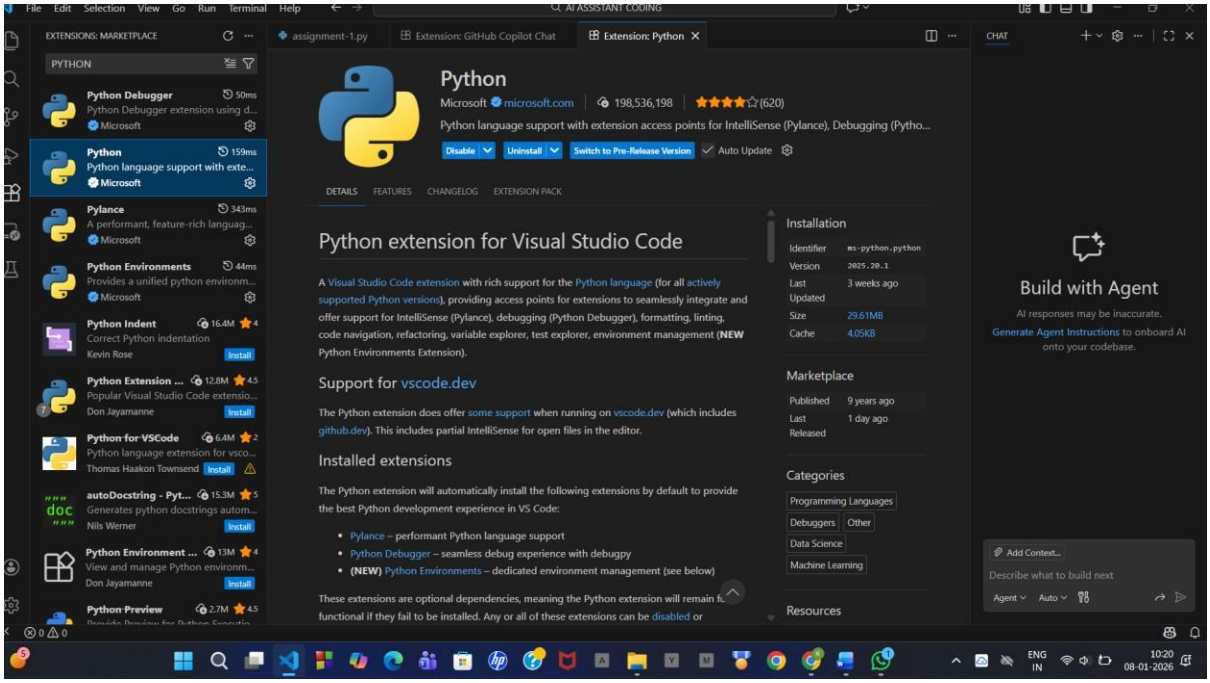
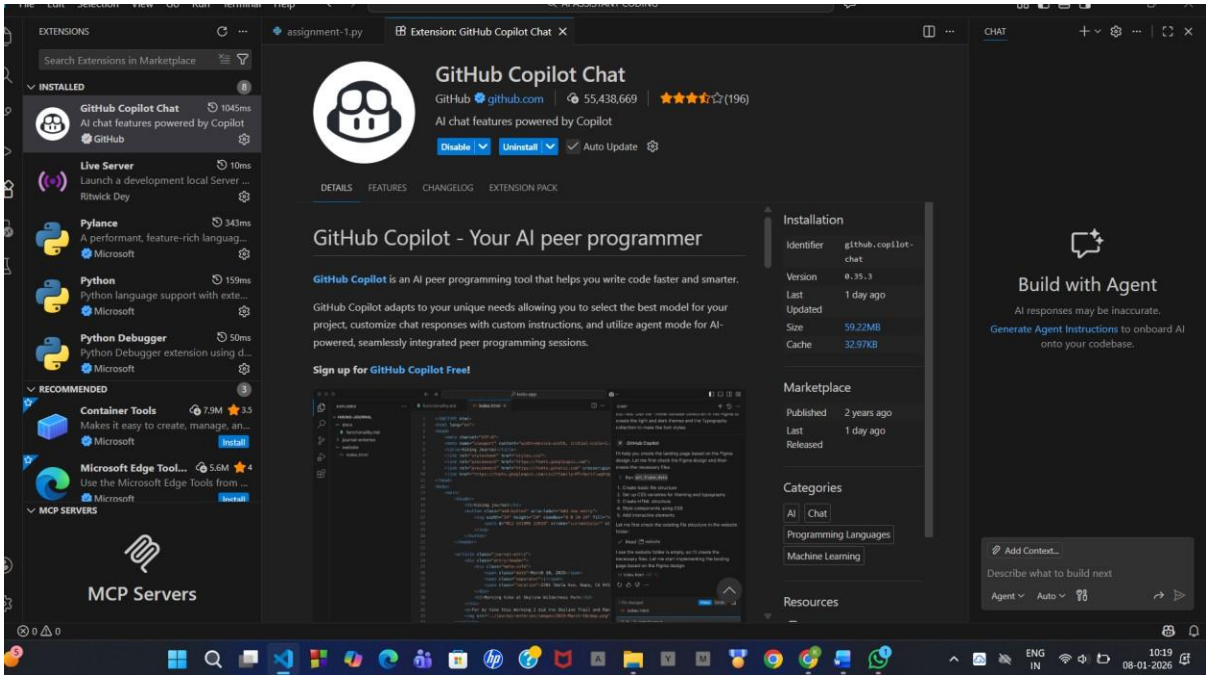
AI-ASSISTED CODING

ASSIGNMENT-1

2303A51242 BATCH-05

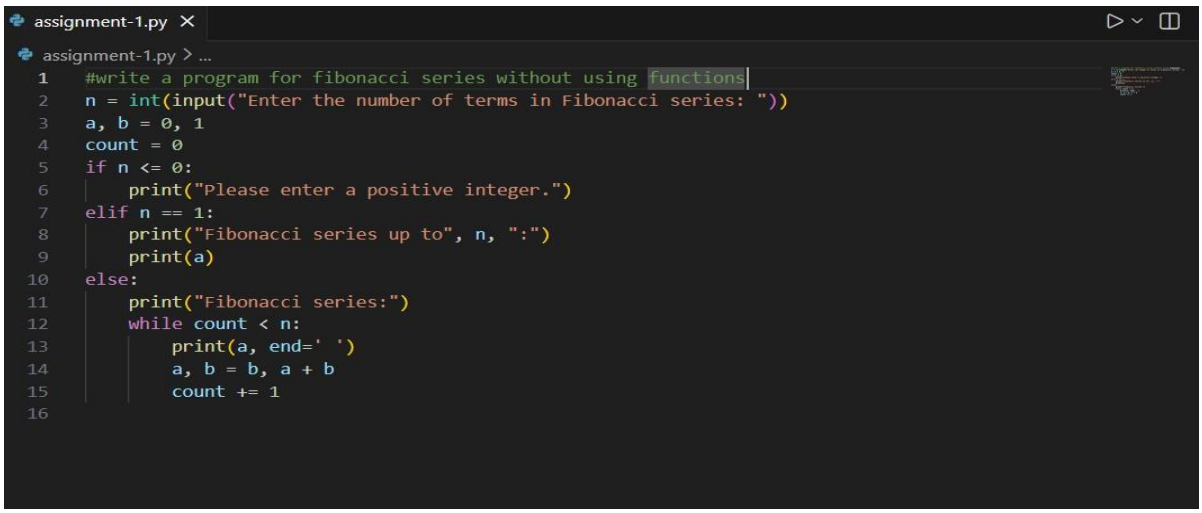
TASK-0:

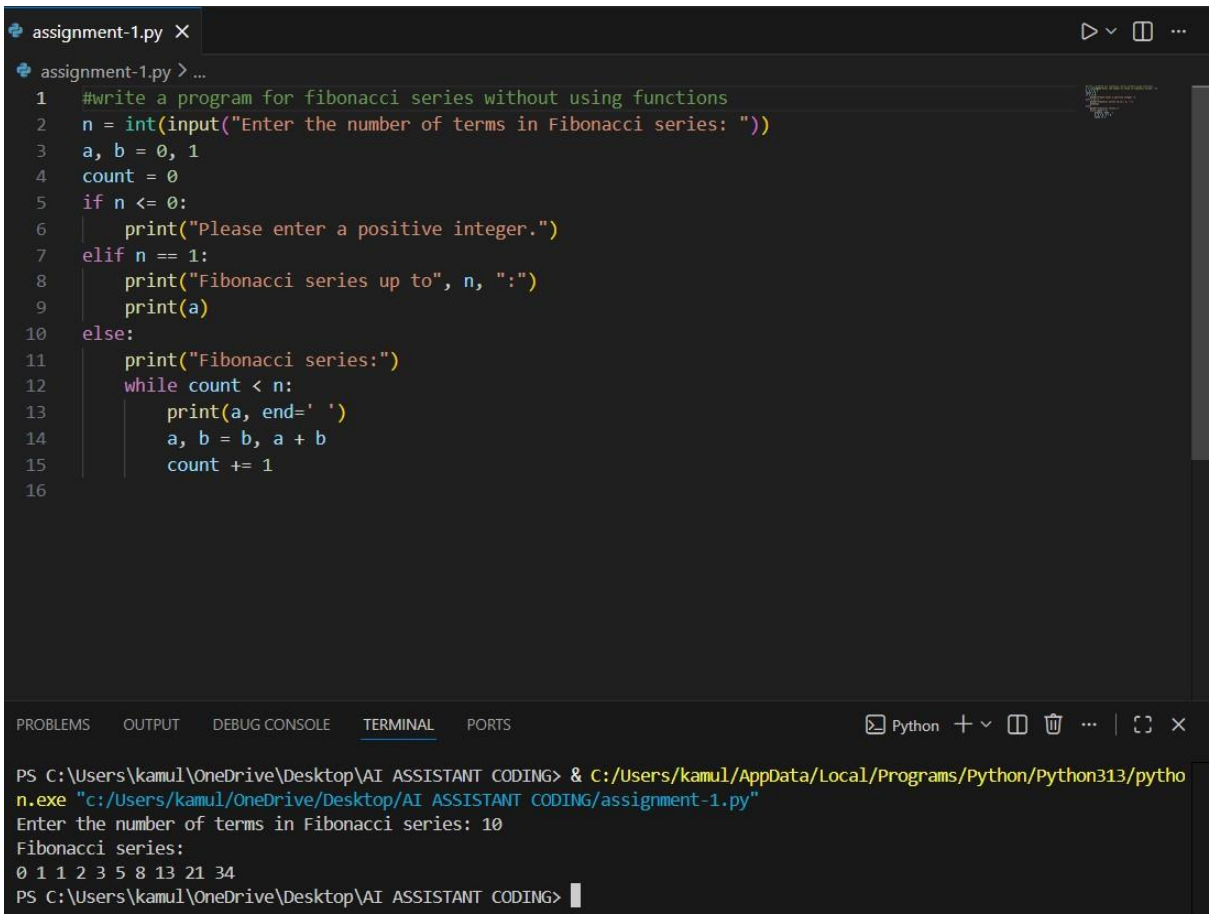




TASK-01:

#write a program for fibonacci series without using functions





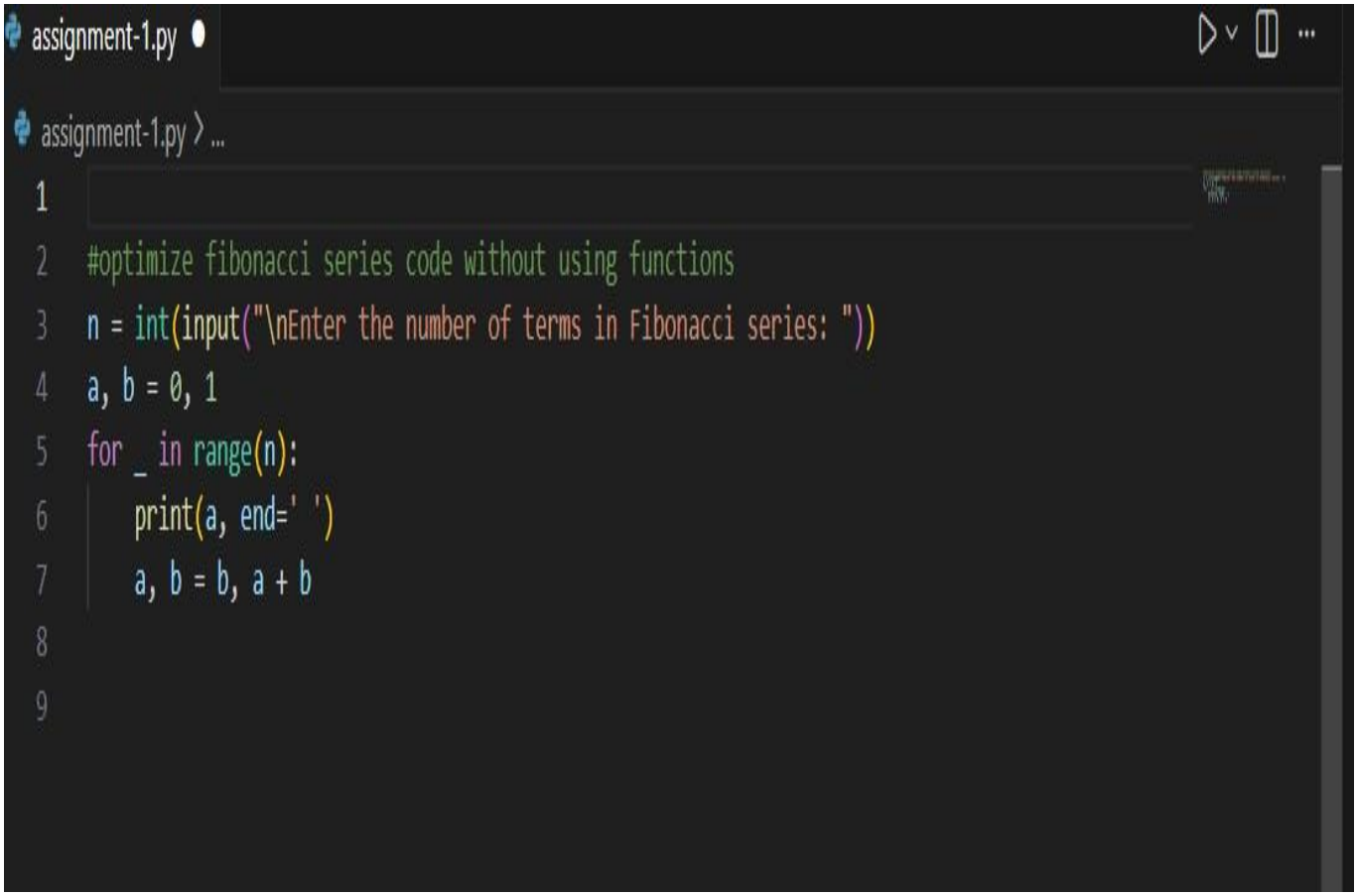
EXPLANATION:

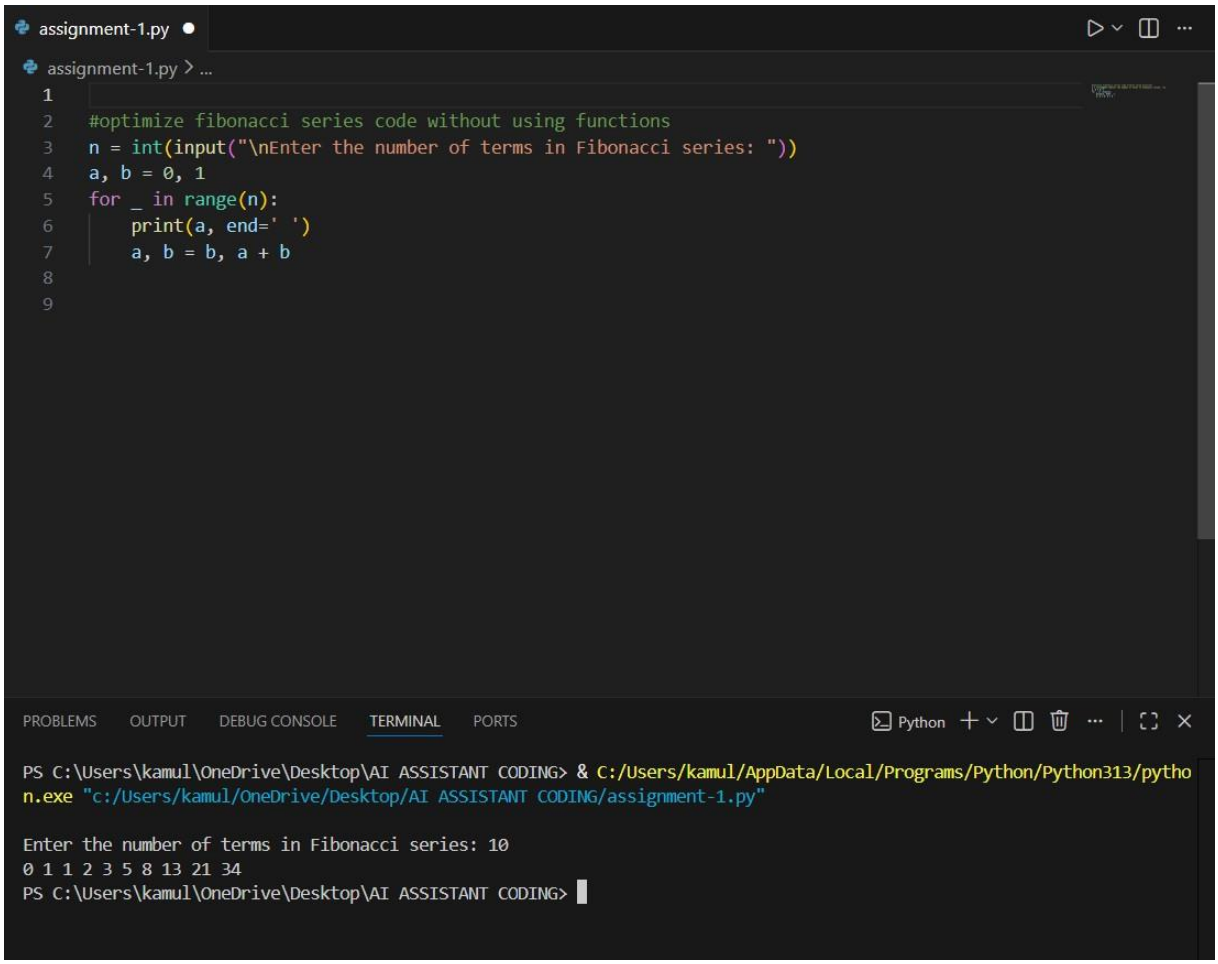
This program generates the Fibonacci series without using any functions by using variables, conditional statements, and a while loop. The user enters the

number of terms to be printed. Two variables a and b are initialized with values 0 and 1 to represent the first two Fibonacci numbers. If the user enters a non-positive number, the program displays an error message. If the input is valid, the program uses a while loop to repeatedly print the current Fibonacci number and update the next value by adding the previous two numbers. This approach demonstrates how the Fibonacci series can be generated iteratively using basic programming constructs.

TASK-02:

#optimize fibonacci series code without using functions



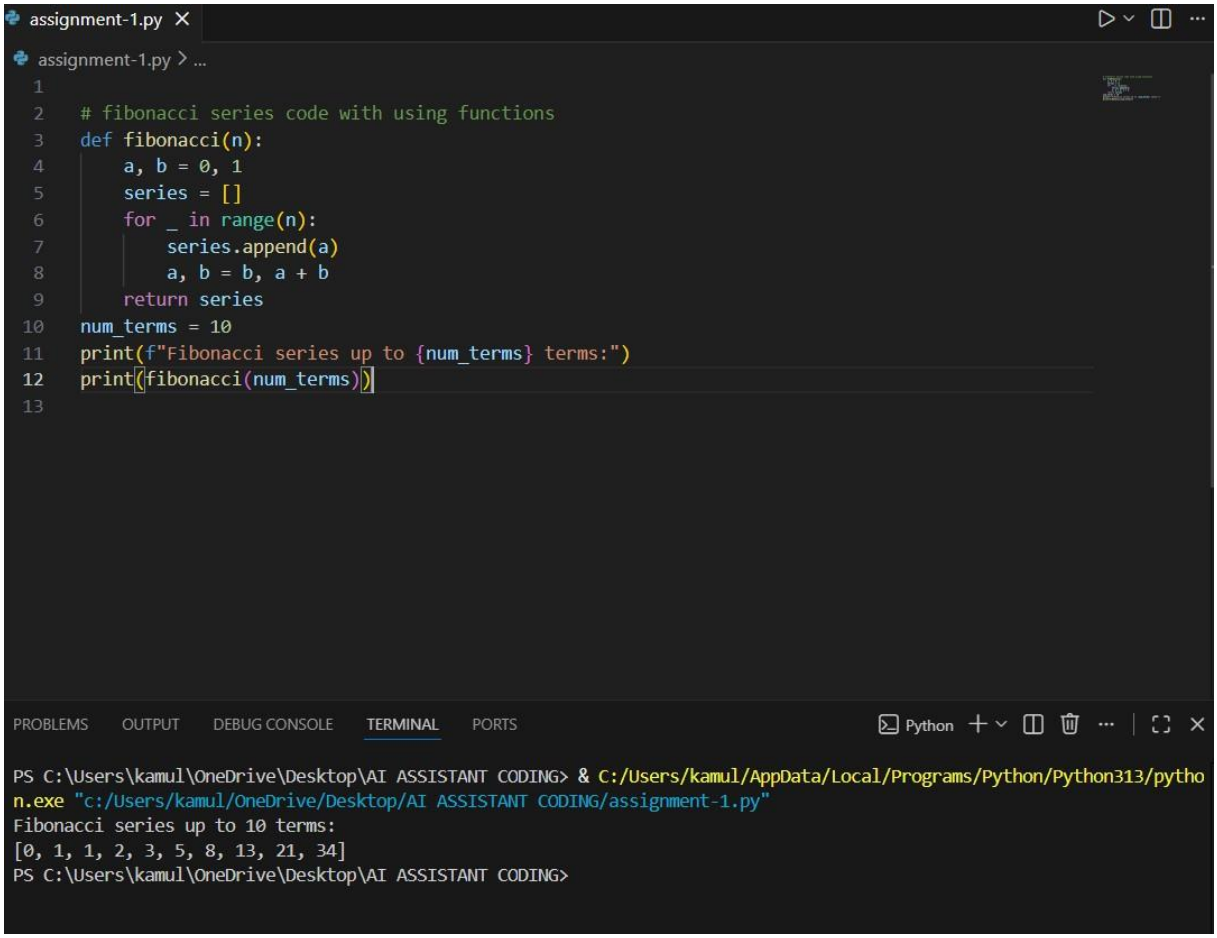


EXPLANATION:

This program prints the Fibonacci series in an optimized and efficient way without using functions. The user enters the number of terms to be displayed. The variables a and b are initialized to 0 and 1, representing the first two Fibonacci numbers. A for loop runs n times, and in each iteration the current value of a is printed. Then, the next Fibonacci number is calculated by updating a and b as the sum of the previous two numbers. This approach avoids unnecessary condition checks and uses minimal variables, making the program simple and efficient.

TASK-03:

fibonacci series code with using functions



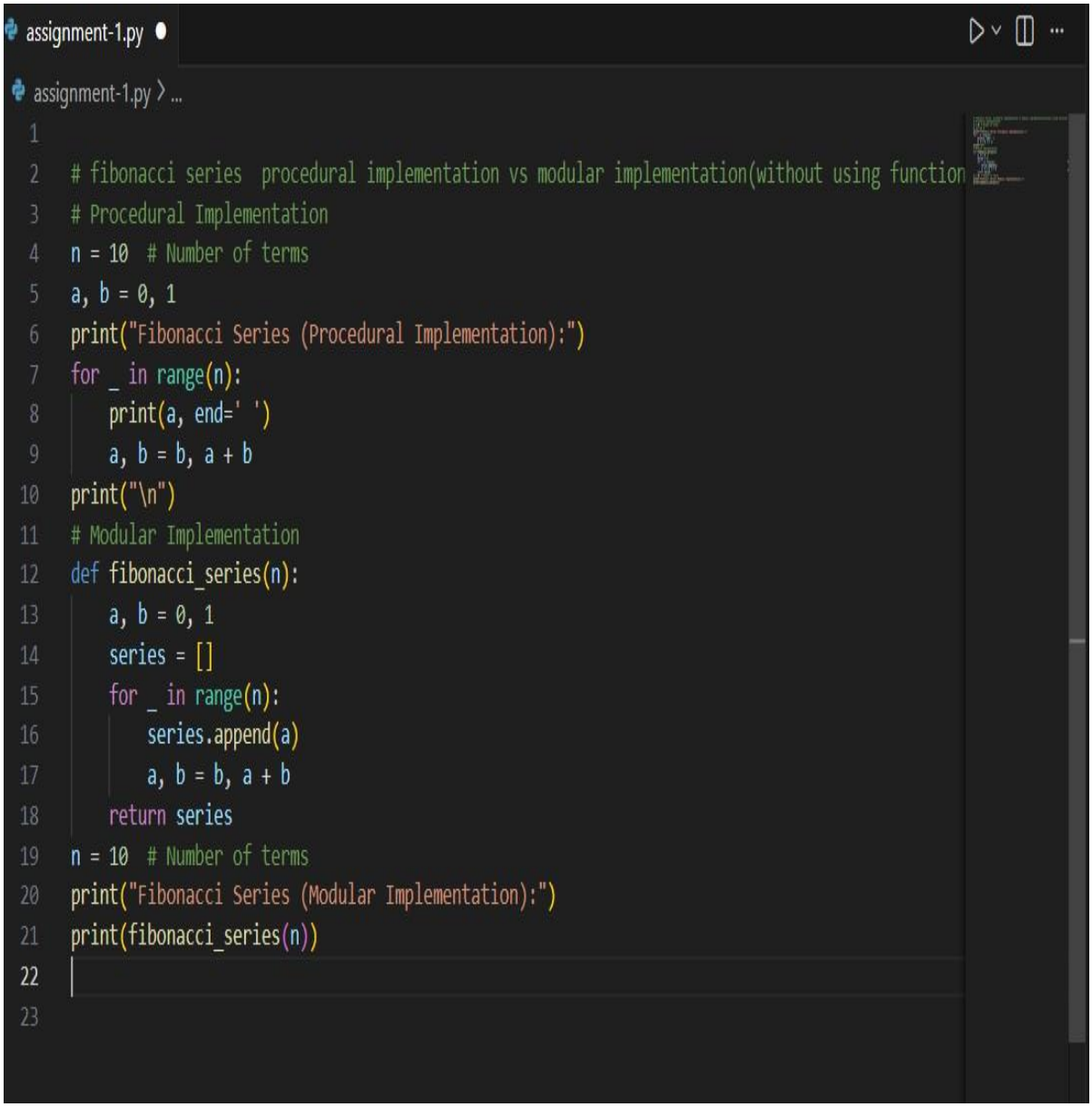
EXPLANATION:

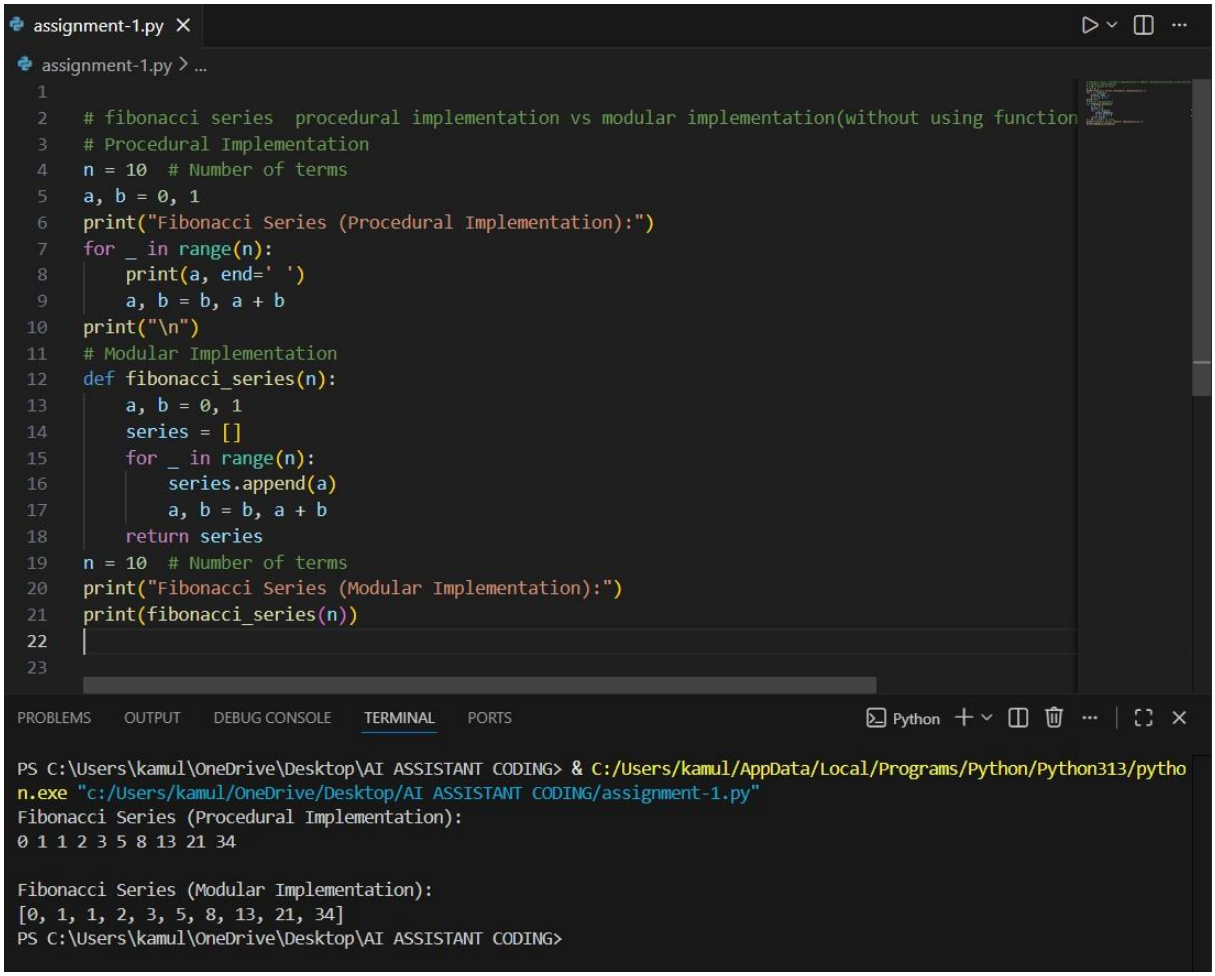
This program generates the Fibonacci series using a function. The function fibonacci(n) initializes the first two Fibonacci numbers as 0 and 1 and uses a for loop to compute the series up to n terms. Each Fibonacci number is stored in a list called series. After calculating all terms, the function returns the list of

Fibonacci numbers. In the main part of the program, the number of terms is set to 10, and the function is called to display the Fibonacci series. This approach improves code readability, reusability, and organization by using functions.

TASK-04:

 # fibonacci series procedural implementation vs modular implementation(without using functions vs with functions)



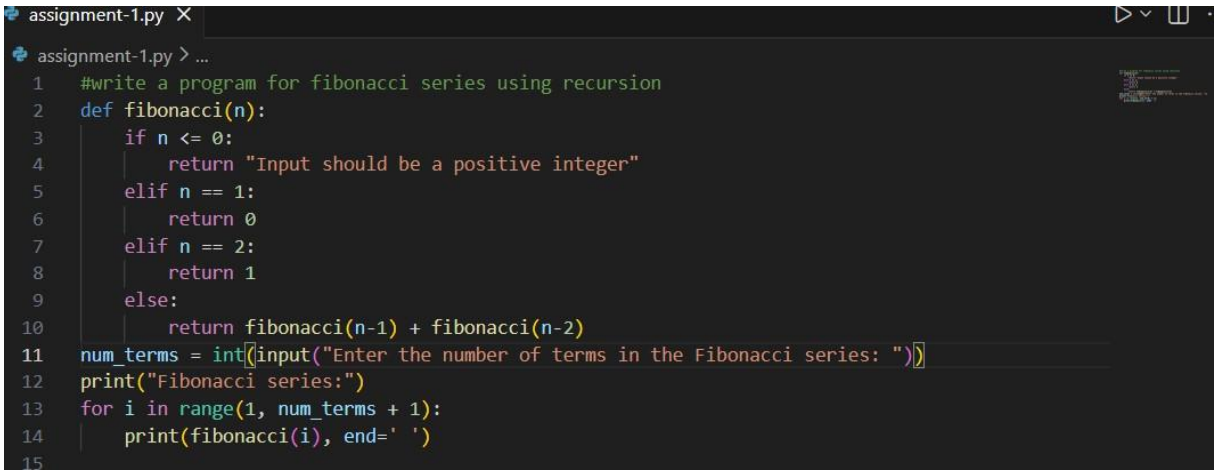


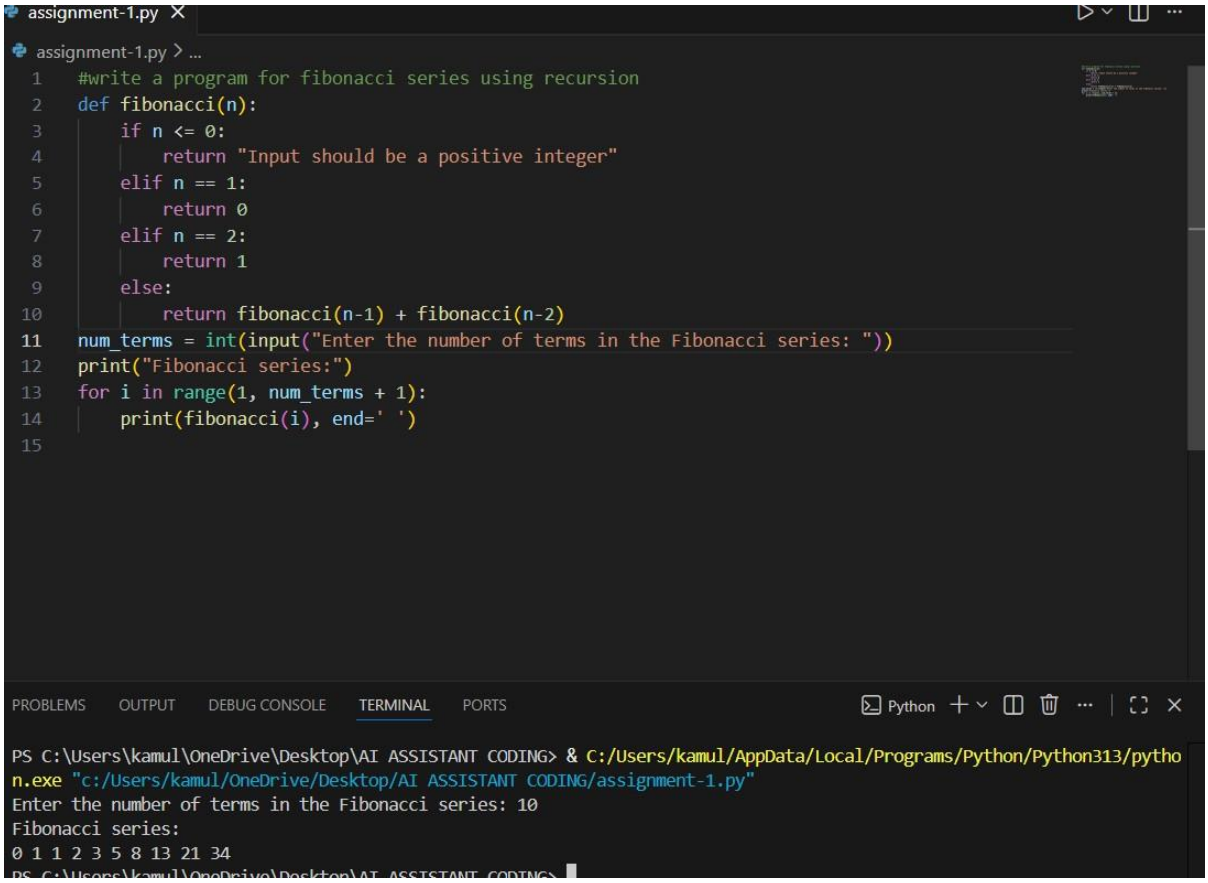
EXPLANATION:

This program demonstrates two different approaches to generating the Fibonacci series: procedural implementation and modular implementation.In the procedural approach, the logic for generating the Fibonacci series is written directly in the main program without using any functions. The variables a and b are initialized to 0 and 1, and a for loop is used to print the Fibonacci numbers one by one. This method is simple but less reusable.In the modular approach, the Fibonacci logic is placed inside a function called fibonacci_series(n). The function computes the Fibonacci numbers using a loop, stores them in a list, and returns the list. The main program then calls this function to display the series. This approach improves code organization, readability, and reusability.Overall, the program highlights the difference between procedural and modular programming by showing how the same problem can be solved in two structured ways.

TASK-05:

#write a program for fibonacci series using recursion





EXPLANATION:

This program generates the Fibonacci series using a recursive function. The function fibonacci(n) returns the *n-th* Fibonacci number. Base conditions are used to return 0 when n = 1 and 1 when n = 2, ensuring the recursion stops. For values greater than 2, the function calls itself to calculate the sum of the

previous two Fibonacci numbers. The user enters the number of terms, and a loop prints the Fibonacci series up to that number. This program demonstrates the concept of recursion by solving a problem through repeated function calls.